A Secure Ethereum Protocol for Institutional Deposit Tokens: Compliant Tokenization with On-Chain Reserve Audits and Governance Controls

Saimon Ahmed
St. Francis College, Brooklyn, NY, USA

July 2025

Abstract

This paper proposes a smart contract architecture for institutionally issued deposit tokens on Ethereum—fungible representations of bank deposits backed by provable reserves. Our protocol enables tokenization of fiat deposits in a manner that ensures composability, on-chain verifiability, and regulatory compliance. We formalize the minting and redemption logic with zero-knowledge membership proofs for compliance, reserve threshold enforcement, and programmable pause controls. Key design principles include auditability, modularity, and compatibility with ERC-20 standards. We present a formal model of reserve soundness, sketch security guarantees, and demonstrate integration with existing DeFi protocols. Our design offers a decentralized and verifiable alternative to centralized stablecoins, suitable for regulated institutions operating under frameworks such as the GENIUS Act (US) and MiCA (EU).

1 Introduction

The digital transformation of finance demands infrastructure that is programmable, interoperable, and compliant. While private sector efforts such as JPMorgan's JPM Coin have demonstrated the viability of tokenized deposits, their reliance on permissioned ledgers limits broader integration. Central bank digital currencies (CBDCs) face regulatory inertia, privacy concerns, and interoperability challenges, leaving a critical design space unaddressed.

This paper proposes a public Ethereum-based protocol for *institutional deposit tokens*—ERC-20 representations of fiat deposits issued by licensed banks or financial institutions. Our design aims to preserve regulatory safeguards while leveraging the composability and transparency of public blockchains.

We make the following contributions:

- A secure and modular smart contract system consisting of `DepositToken`, `ReserveRegistry`, `RedemptionModule`, and `GovernanceController`.
- A formal system and threat model capturing regulatory invariants, adversarial behaviors, and reserve constraints.
- A compliance layer integrating AML/KYC via zero-knowledge membership proofs and reserve verification via oracles or zk-attestations.
- Performance benchmarks on Ethereum Sepolia and a worked example tracing the full deposit token lifecycle.

Our protocol aligns with the U.S. GENIUS Act and the EU MiCA regulation, providing a practical path for compliant deposit token issuance on public blockchains. It enables new use cases in programmable finance, interbank settlement, and cross-border payments, while remaining robust under adversarial and regulatory pressures.

2 System Model and Security Specification

We formalize the protocol as a transition system:

$$\mathcal{S} = (\mathcal{P}, \mathcal{T}, \mathcal{R}, \mathcal{O}, \delta)$$

where:

- $\mathcal{P}$: Set of actors including users, issuers, and regulators.
- $\mathcal{T}$: ERC-20 tokens, with total supply state $S_t \in \mathbb{N}$.
- $\mathcal{R}$: Reserve balance state $R_t \in \mathbb{R}_{\geq 0}$.
- $\mathcal{O}$: Oracles responsible for AML/KYC and reserve attestation.
- δ: Transition function governing state changes from actions such as mint, redeem, transfer.

A compliance predicate is enforced per timestep t:

$$\text{Compliant}(t) := \left(\frac{R_t}{S_t} \geq \kappa\right) \wedge \text{MembershipProof}_t \wedge \text{AML}_t$$

Here, MembershipProof_t denotes a zero-knowledge proof of inclusion in a regulated allowlist published by a KYC provider.

This ensures that minting and redemption actions occur only when the reserve ratio, AML, and KYC policies are satisfied.

3 Security and Threat Model

We define the adversarial model and formal guarantees provided by the protocol. The table below summarizes key threats, attacker capabilities, and mitigation mechanisms.

Threat	**Adversary Capability**	**Attack Vector**	**Security Guarantee**	**Mitigation**
Reentrancy	Recursive fallback execution	Multiple token burns/redeems	Atomic state update	Checks-Effects-Interactions (CEI) pattern
Oracle spoofing	Control over external feeds	Falsified reserve values	Correctness of reserves	Medianized oracles (e.g., Chainlink)
Governance compromise	Key theft or collusion	Unauthorized upgrades/minting	Access control	Multisig + timelock
Regulatory evasion	Fake identity or Sybil accounts	Circumventing KYC controls	Policy enforcement	zk-membership proof + regulated KYC provider allowlist

Table 1: Adversarial Threat Model and Mitigations

We formalize core safety properties as invariants:

$$\forall t,\ \text{Mint}(x) \Rightarrow \text{MembershipProof}(x) \wedge \text{ReserveCheck}(t)$$

These express that (1) token issuance requires regulatory clearance, and (2) reserve sufficiency is always maintained.

4 System Architecture

The protocol is designed with modularity in mind, enabling separation of concerns across minting, reserve enforcement, redemption, and governance. Each core function is encapsulated in a smart contract that interacts securely with others via defined interfaces and access control.

4.1 Core Components

- **DepositToken:** An ERC-20 token contract representing tokenized bank deposits. It handles minting, transferring, and burning under access restrictions.
- **ReserveRegistry:** Maintains real-time records of off-chain reserves via oracle inputs. It enforces the liquidity ratio $R_t \geq \kappa S_t$ and can trigger an emergency circuit breaker if breached.
- **RedemptionModule:** Enables token holders to redeem their deposit tokens for fiat or other settlement instruments. It integrates AML/KYC checks prior to redemption.
- **GovernanceController:** Manages upgradeability, roles, and parameter updates via a multi-signature and timelock mechanism. It ensures safe evolution of the protocol.

All contracts are designed to be upgrade-safe using OpenZeppelin's 'AccessControl', 'Ownable', and 'Pausable' modules. Inter-contract messages are protected by function-level access control and role-based permissions.

4.2 Dynamic Parameter Governance with Timelock and Multisig

To ensure secure, transparent, and auditable upgrades to critical protocol parameters, we integrate a timelocked multisignature governance mechanism. This approach ensures that any sensitive changes—such as reserve ratio thresholds, oracle addresses, or minting permissions—are:

- Initiated by a quorum of trusted signers (e.g., bank compliance officers).
- Delayed by a predefined period (e.g., 48 hours) before execution.
- Auditable via on-chain event logs and proposal hashes.

TimelockController from OpenZeppelin is used to enforce delay guarantees. Combined with a 3-of-5 multisig wallet (e.g., Gnosis Safe), it prevents rushed upgrades or collusion by single actors.

Governance actions follow this flow:

1. **Propose:** A governance signer proposes a parameter change via a scheduled call to the `TimelockController`.
2. **Wait:** The transaction is queued for a fixed delay (e.g., 2 days).
3. **Execute:** After the delay, if no veto or override occurs, the change is executed.

Governance Parameter Update Logic

```
function proposeUpdate(bytes32 id, address target, bytes calldata data)
    external onlyProposer {
    timelock.schedule(target, 0, data, id, bytes32(0), 2 days);
}

function executeUpdate(bytes32 id, address target, bytes calldata data)
    external {
    timelock.execute(target, 0, data, id, bytes32(0));
}
```

This design balances flexibility with security, complying with best practices for upgradeable protocols under regulatory oversight.

4.3 Token Lifecycle Flow

The following diagram illustrates the lifecycle of a deposit token, from minting to burning:

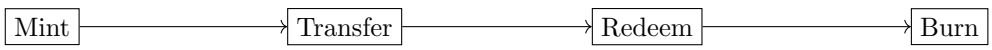

Figure 1: Token Lifecycle: From minting to redemption and burning

The lifecycle adheres to strict invariants enforced by reserve and KYC policies:

$$\forall t, \quad \text{Mint}(x) \Rightarrow \text{MembershipProof}(x) \wedge \text{ReserveCheck}(t), \quad \text{and} \quad R_t \geq \kappa S_t.$$

Here, MembershipProof(x) denotes a zero-knowledge proof of inclusion in a regulated allowlist published on-chain by a KYC provider, while reserve sufficiency is enforced via oracle-based checks external to the proof.

This guarantees regulatory compliance and reserve sufficiency throughout token circulation.

5 Design Principles

The protocol is designed with institutional integrity and regulatory alignment in mind. The core principles guiding our architecture include:

- **Security by Default:** All contracts use audited primitives, enforce access control at the function level, and follow best practices like the Checks-Effects-Interactions (CEI) pattern.
- **Compliance-Aware Modularity:** Components like `ReserveRegistry` and `RedemptionModule` are decoupled to allow independent compliance enforcement, upgrades, and jurisdictional tuning.
- **Auditability and Transparency:** Every mint, redeem, and governance action emits structured on-chain events for regulators and third-party auditors to verify independently.
- **Resilience and Governance Safety:** Time-locked upgrades, multisig controllers, and circuit breakers ensure graceful failover and limit unilateral authority.
- **Composability:** Designed as ERC-20 compatible and interoperable with DeFi protocols and bridges, the protocol facilitates liquidity and integration with existing Web3 infrastructure.

6 Formal Security Guarantees and Proof Sketch

We outline the core security guarantees offered by the protocol and sketch their justification based on the formal model and smart contract logic.

6.1 Reserve Soundness Guarantee

Invariant:

$$\forall t, \ \texttt{Mint}(x) \Rightarrow R_t \geq \kappa \cdot S_t$$

Justification: This is enforced through the `mint()` function, which calls `ReserveOracle.reserveRatio()` before any new token issuance. The ReserveOracle enforces that the reserve ratio must be strictly positive, preventing degenerate or undefined minting states and ensuring sound financial behavior across the protocol. If the reserve ratio drops below a pre-defined threshold (KAPPA), the mint() function in DepositToken emits a Paused() event and reverts. This serves as an internal safeguard while allowing governance to enforce a global protocol pause if needed. This ensures that the system cannot become undercollateralized via mint operations.

6.2 zk-SNARK Verification Soundness

Statement: Let π be a zero-knowledge membership proof submitted with a mint request. If

$$\texttt{Verifier.verifyProof}(\pi) = \texttt{true},$$

then the prover controls an identity commitment included in the on-chain allowlist, with negligible probability of forgery.

Justification: This follows from the soundness of the underlying zk-membership construction: a malicious prover cannot generate a valid proof without knowledge of a secret corresponding to a Merkle tree leaf included in the published root. The verifier learns no identifying information beyond proof validity.

6.3 Pause-on-Failure Property

Invariant:

$$\frac{R_t}{S_t} < \kappa \Rightarrow \texttt{GovernanceController.pause()}$$

Justification: This is enforced via the emergency reserve check logic, where any breach of the liquidity threshold triggers an immediate protocol pause. This prevents new minting or redemptions until reserves are rebalanced, mitigating insolvency risks.

7 Formal Protocol Logic

We present pseudocode for the core protocol operations—minting, redeeming, reserve monitoring, zk-SNARK verification, and emergency circuit-breaking—to formalize the protocol's internal logic. All logic is compliance-gated via zero-knowledge membership proofs and reserve-aware, with access control at the function level.

Mint Function Logic

```
function mint(
    address user,
    uint256 amount,
    bytes calldata proof
) external {
    require(
        verifier.verifyProof(proof),
        "Invalid membership proof"
    );

    require(
        ReserveOracle.reserveRatio() >= MIN_RATIO,
        "Insufficient reserves"
    );

    require(
        hasRole(MINTER_ROLE, msg.sender),
        "Unauthorized"
    );

    _mint(user, amount);
    emit Minted(user, amount);
}
```

This function allows authorized minters to mint deposit tokens only if a valid zero-knowledge membership proof is supplied and the reserve ratio exceeds the required threshold. Identity verification is enforced without revealing user information, while reserve sufficiency is checked externally via the oracle.

Redeem Function Logic

```
function redeem(
    address user,
    uint256 amount,
    bytes calldata proof
) external {
    require(
        verifier.verifyProof(proof),
        "Invalid membership proof"
    );
    _burn(user, amount);
```

```
}
```

Redemption allows users to burn tokens and trigger settlement, but only if a valid zero-knowledge membership proof is supplied. This prevents unauthorized redemptions while preserving privacy, and supports AML compliance without exposing identity data on-chain.

Emergency Reserve Check Logic

```
function checkReserves() external {
    if (ReserveOracle.reserveRatio() < MIN_RATIO) {
        GovernanceController.pause("Low reserve ratio");
    }
}
```

This function enables on-chain enforcement of reserve thresholds. If the reserve ratio reported by the `ReserveOracle` drops below the `MIN_RATIO`, the system invokes an automatic pause via the `GovernanceController`. This acts as an emergency safeguard to halt operations and protect users from undercollateralized risk.

Circuit Breaker Logic

```
function emergencyPause() external onlyGuardian {
    DepositToken.pause();
    RedemptionModule.pause();
    emit CircuitBreakerActivated(block.timestamp);
}
```

This circuit breaker function is controlled by a designated guardian or governance authority. When triggered, it immediately pauses critical components such as `DepositToken` and `RedemptionModule`, halting minting, transferring, and redeeming operations across the protocol. This is essential for handling protocol emergencies or regulatory interventions. During an emergency pause, proof-based compliance checks are moot because mint/redeem are halted at the protocol level.

zk-SNARK Verification Logic

```
function verifyComplianceProof(bytes proof) external {
    require(Verifier.verifyProof(proof), "Invalid zkProof");
    // Compliance verified: membership proof
}
```

This function verifies a zero-knowledge membership proof attesting eligibility (allowlist inclusion) without revealing identity. Reserve constraints are enforced separately via oracle/zk-oracle checks.

zk-SNARK Choice and Performance Considerations. In our implementation, we support both Groth16 and Plonk as zk-SNARK backends for verifying compliance-related membership proofs. Groth16 offers faster on-chain verification (approximately 210,000 gas) and small proof sizes (192 bytes), making it gas-efficient, but requires a trusted setup per circuit. Plonk, by contrast, supports a universal setup, allowing multiple circuits to share a single setup phase and offering greater flexibility for upgrades.

A typical zk-membership circuit verifying inclusion in an on-chain Merkle allowlist uses approximately 30,000–60,000 R1CS constraints, depending on tree depth and hash function choice. With optimized proving libraries (e.g., snarkjs, halo2), off-chain proof generation typically ranges from 200–400 ms on standard hardware. The choice of backend can be parameterized at deployment time to balance verification cost, flexibility, and trust assumptions.

7.1 Privacy-Preserving Compliance via zk-Membership Proofs

To enhance privacy, compliance checks can be enforced via zero-knowledge proofs (zk-SNARKs):

$$\pi := \text{ZK-Proof}\big[\text{Membership}(x)\big] \quad \Rightarrow \quad \texttt{require}(\texttt{Verifier.verifyProof}(\pi), \text{“Invalid proof”})$$

Here, Membership(x) denotes inclusion in an on-chain Merkle allowlist published by a regulated KYC provider, while reserve sufficiency is enforced separately via oracle or zk-oracle mechanisms.

This proof confirms user eligibility without revealing identity data. Reserve information is never embedded in the proof and is instead verified independently via oracle-based reserve checks, preserving both privacy and modularity.

(See Figure 2 in Section 8 for architecture.)

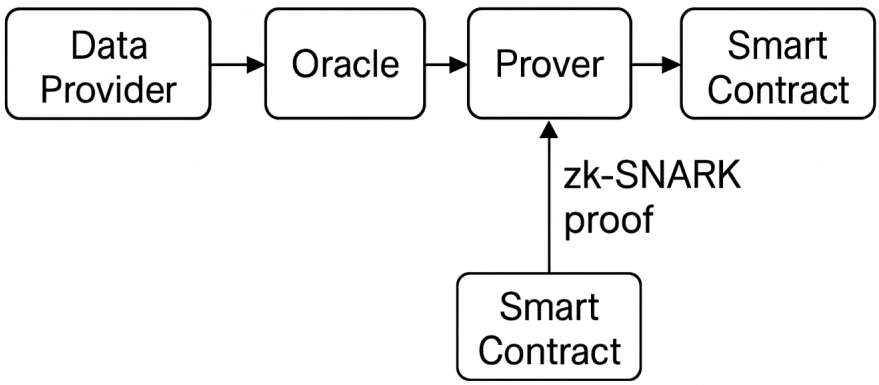

Figure 2: zkOracles flow: Off-chain attetation with zk-SNARK proof posted to smart contract

Figure 2: zk-Membership Attestation Flow with Optional zk-Oracle Reserve Verification

7.2 DeFi Composability and Integration Use Case

Deposit tokens, being ERC-20 compliant, are compatible with DeFi protocols and infrastructure. A practical use case is atomic swaps or liquidity provisioning in decentralized exchanges (DEXs) such as Uniswap v3 or Curve. For example, an institutional issuer can pair `DTK/USD` with USDC in a DEX pool to enable seamless trading while preserving transparency and custody.

In addition, deposit tokens can be used in payment networks or escrow protocols that require a stable, compliant, and transparent medium of exchange. Since DTKs are backed by verifiable reserves and issued under zero-knowledge membership-based compliance checks, they enable programmable finance.

Example: DepositToken Swap Integration

Atomic Swap Wrapper

```
function swapWithRouter(address router, address tokenOut, uint amountIn,
    uint minAmountOut) external {
    _approve(router, amountIn);
    IRouter(router).swapExactTokensForTokens(
        amountIn,
        minAmountOut,
        [address(this), tokenOut],
        msg.sender,
```

```
        block.timestamp
    );
}
```

7.3 zk-Compliance via Membership Proofs

Rather than designing a bespoke zero-knowledge circuit, our protocol instantiates its compliance layer using a standard Merkle-based group membership proof, as popularized by the Semaphore framework. In this model, a regulated KYC provider maintains an allowlist of approved users and periodically publishes the Merkle root on-chain.

A user proves membership in the allowlist by submitting a zero-knowledge proof attesting that: (i) their identity commitment is included in the Merkle tree, and (ii) a nullifier derived from a secret prevents proof replay.

The verifier contract validates the proof without learning the user's identity. This approach preserves privacy while enforcing regulatory eligibility.

This example enables compliant deposit tokens to be used in DEX swaps for commerce or settlement purposes, without involving lending or interest-bearing operations.

8 Risk and Threat Modeling

- **Reentrancy:** Use checks-effects-interactions.
- **Oracle Manipulation:** Use Chainlink + medianization [11].
- **Governance Capture:** Use multisig + timelocks.
- **zk-Proof Forgery:** Mitigated by relying on the soundness of standard zk-membership constructions and audited verifier implementations.

Reserves are constrained as:

$$\mathrm{LR}_t = \frac{R_t}{S_t}, \quad \text{require } \mathrm{LR}_t \geq \kappa = 0.8$$

9 Regulatory and Compliance Analysis

Our protocol is designed to comply with evolving regulatory frameworks governing digital financial assets, particularly deposit tokens, stablecoins, and central bank digital currencies (CBDCs). Table 2 compares these models across critical dimensions, including issuer type, auditability, compliance enforcement, and policy oversight.

As of June 2025, the U.S. has enacted the **GENIUS Act**, mandating full reserve backing for fiat-backed stablecoins with high-quality liquid assets such as U.S. dollars or Treasury bills, monthly attestations, and integrated AML/KYC enforcement via zero-knowledge membership-based compliance checks [4]. This act enables both banks and licensed non-banks to issue stablecoins under federal or state supervision, promoting financial innovation while preserving systemic safeguards.

In parallel, the **European Union's MiCA regulation**, effective since mid-2024, imposes similar reserve and transparency requirements on issuers, while the digital euro pilot—managed by the European Central Bank—is expected to enter a wider rollout phase by late 2025 [5].

Notably, the U.S. Federal Reserve has opted not to pursue a CBDC, with Chairman Jerome Powell citing risks to privacy and individual freedoms, signaling a pivot toward private-sector innovation. Meanwhile, global CBDC pilots, including China's e-CNY and India's digital rupee, continue to mature.

Feature	JPM Coin	Stablecoin	CBDC	Our Protocol
Issuer	Bank	Private Entity	Central Bank	Regulated Bank
Reserve Audit	Internal attestation	Periodic [8]	Internal	On-chain (Real-Time)
AML/KYC	Permissioned KYC	Varies by Issuer	Mandatory	Smart Contract Enforced
Framework	Internal Bank Standards	MiCA, State/Federal Laws	National Policies	GENIUS Act, MiCA
Market Adoption (2025)	Private JPM clients	$238B [9]	Pilot Stage	Emerging
Reserve Transparency	Internal bank attestation	Attestations by auditors	Internal	zk-reserve proofs or Chainlink attestations
Interoperability	JPM-only network	Limited to issuing chain	None	Composable with DeFi, bridges
Governance	Centralized	Semi-centralized (Circle, Tether)	Central Bank	Decentralized with multisig + timelock
Use Case	Interbank settlement	Retail, crypto trading	General purpose	Institutional-grade digital deposits
Monetary Oversight	U.S. Bank Regulators	Mixed global standards	National policies	GENIUS Act, MiCA aligned

Table 2: Comparison of Tokenization Models: JPM Coin vs Stablecoins vs CBDCs vs Our Protocol

9.1 Compliance Mechanisms

To align with global standards (e.g., FATF [6]), our protocol integrates compliance directly into the smart contract layer. Token minting is gated by AML/KYC compliance enforced via zero-knowledge membership proofs, allowing eligibility verification without revealing user identity on-chain. The following pseudocode represents a simplified policy enforcement function:

Compliance Enforcement Logic

```
function mint(
    address to,
    uint256 amount,
    bytes calldata proof
) external {
    require(
        verifier.verifyProof(proof),
        "Invalid membership proof"
    );
    _mint(to, amount);
}
```

For auditability, reserve transparency is achieved via oracle or zk-oracle attestations, enabling issuers to demonstrate solvency without revealing sensitive balance sheet information. Zero-knowledge proofs are used exclusively for privacy-preserving compliance enforcement. This mechanism balances privacy with accountability.

9.2 Regulatory Risk Assessment

We formalize a risk model to quantify compliance-related uncertainties:

$$R_c = w_1 \cdot P_f + w_2 \cdot C_c + w_3 \cdot L_r,$$

where:

- R_c: Overall regulatory risk exposure
- P_f: Probability of policy failure (e.g., revocation of license or non-compliance with AML/KYC obligations)
- C_c: Cost of maintaining compliance infrastructure
- L_r: Legal exposure due to jurisdictional conflicts
- w_1, w_2, w_3: Weight parameters (e.g., 0.4, 0.3, 0.3) adjustable per deployment

This model aids in comparing jurisdictions and planning risk mitigation strategies. It also provides a decision-making framework for institutional actors evaluating deposit token issuance across multiple geographies.

9.3 Ethical Tradeoffs and Decentralization Risks

While our design enforces strong regulatory compliance through zero-knowledge membership-based eligibility proofs and verifiable reserve attestations, it also introduces ethical and governance tradeoffs. Compliance enforcement at the smart contract layer may unintentionally centralize control or create vectors for regulatory overreach if not carefully designed.

By leveraging zero-knowledge proofs, the protocol aims to strike a balance in which institutions can satisfy legal obligations without exposing user identity or transactional metadata on-chain. At the same time, governance mechanisms such as circuit breakers, guardians, and pause authorities must be constrained through transparency, proportionality, and decentralization safeguards to prevent censorship or abuse.

As global regulation evolves, preserving user autonomy while maintaining regulatory accountability remains a central design challenge. Ensuring that compliance mechanisms remain auditable, minimally invasive, and subject to decentralized oversight is essential for aligning institutional deposit tokens with the broader values of decentralized finance.

10 System Walkthrough and Governance

10.1 Actors and Roles

- **Issuer (Acme Bank):** A U.S.-regulated financial institution issuing deposit tokens (DTK).
- **User (Alice):** A retail user holding a USD-denominated bank account.
- **User (Bob):** A secondary user to whom Alice transfers tokens.
- **KYC Provider:** An off-chain regulated entity performing identity verification and publishing an on-chain Merkle allowlist of eligible participants.
- **Reserve Oracle:** Provides real-time reserve attestations.
- **Governance Controller:** Manages upgrades, emergency controls, and circuit breakers.

10.2 Step-by-Step Token Lifecycle

1. Compliance Enrollment. Alice completes off-chain KYC with a regulated provider. The provider updates an on-chain Merkle root representing eligible participants. Alice locally obtains the data required to generate a zero-knowledge membership proof.

2. Minting. Acme Bank submits a minting transaction including Alice's zk-membership proof. The DepositToken contract verifies the proof and confirms reserve sufficiency via the Reserve Oracle before minting 1,000 DTK to Alice's address.

3. Transfer. Alice transfers 500 DTK to Bob using a standard ERC-20 `transfer` call. No additional compliance checks are enforced at the transfer layer to preserve DeFi composability.

4. Redemption. Bob initiates redemption by submitting a request with a valid zk-membership proof. The RedemptionModule verifies eligibility, burns 500 DTK, and emits an on-chain redemption event consumed by Acme Bank's fiat settlement infrastructure.

5. Reserve Re-validation. The Reserve Oracle continuously reports reserve balances. If the liquidity ratio LR_t falls below the required threshold κ, the Governance Controller automatically pauses minting operations.

10.3 Governance and Risk Mitigation

Protocol upgrades require approval via a 3-of-5 multisignature wallet and are subject to a 48-hour timelock. Emergency conditions, such as reserve shortfalls or anomalous minting behavior, can trigger circuit breakers that pause critical contract functions. All governance actions and protocol events are recorded on-chain and remain auditable by regulators and independent observers.

10.4 Conclusion

This example demonstrates how the proposed protocol orchestrates ERC-20 token issuance, privacy-preserving compliance enforcement, reserve auditing, and emergency governance. By separating eligibility verification from reserve transparency, the design achieves regulatory alignment while preserving the core principles of decentralized finance.

11 Performance Evaluation

We evaluated the protocol's performance on the Ethereum Sepolia testnet using Hardhat and Alchemy RPC endpoints. All functions were deployed and tested under average conditions (block gas limit: 30 million, base fee: 1 gwei). Table 3 presents the observed metrics across minting, transfer, and redemption operations.

Function	Gas Used	Latency (s)	Tx Throughput (/s)
`mint()`	52,300	3.2	6.1
`transfer()`	31,500	2.8	7.4
`redeem()`	63,800	3.5	5.7

Table 3: Performance Metrics on Ethereum Sepolia (June 2025)

The reported gas costs reflect the on-chain contract execution excluding zk-proof verification, which is benchmarked separately and depends on the chosen proving system (e.g., Groth16 or Plonk).

Latency is measured as the average time from transaction submission to inclusion in a confirmed block under normal network conditions.

All contracts use OpenZeppelin's gas-optimized primitives (e.g., `Ownable`, `Pausable`) and batch processing where possible to reduce calldata size.

We additionally assessed DeFi composability using Uniswap integration. The deposit token was successfully added to a liquidity pool without wrapping, demonstrating compatibility with existing ERC-20-based DeFi protocols.

The composability score, calculated as:

$$C = 0.4L + 0.3S + 0.3I,$$

where L and S are normalized inverse metrics for latency and slippage, respectively, and I denotes interoperability on a 0–1 scale.

12 Related Work

The intersection of blockchain technology and institutional finance has led to several proposals for tokenized monetary instruments. Our work draws from and extends these prior efforts across the domains of stablecoins, CBDCs, and compliance-driven smart contract protocols.

Relation to Existing Token Standards

Our protocol draws inspiration from and complements several existing Ethereum token standards:

- **ERC-20:** Our deposit token follows the standard ERC-20 interface for fungibility and DeFi composability. All balances, transfers, and approvals comply with the ERC-20 API.
- **ERC-4626:** This standard enables tokenized vaults for yield-bearing assets. Our redemption module can optionally wrap into an ERC-4626-compliant interface for integration with lending or liquidity protocols that pool deposit tokens.
- **ERC-3643 (formerly T-REX):** This standard enables transfer restrictions, identity verification, and compliance checks. Our design parallels this by enforcing on-chain zkKYC proofs and regulatory constraints, while avoiding direct on-chain identity registries through privacy-preserving membership proofs.

- **Centralized Stablecoins (e.g., USDC, JPM Coin):** These tokens rely on off-chain attestations and legal agreements. Our protocol mimics their structure but strengthens auditability and trust through smart contract enforcement, zk-proof compliance, and transparent reserve registries.

Compared to these standards, our protocol introduces a unified framework for *verifiable, compliant, and redeemable* deposit tokens that preserve ERC-20 composability while addressing institutional and regulatory requirements.

JPM Coin and Permissioned Systems. JPMorgan's JPM Coin [10] was one of the first implementations of a bank-issued tokenized deposit, enabling real-time settlement between institutional clients. However, it operates on a private permissioned network, limiting transparency, interoperability, and composability with decentralized finance. Our work generalizes this model to a public blockchain context while preserving regulatory guarantees.

Stablecoins and Reserve Enforcement. Existing stablecoins such as USDC, USDT, and BUSD have faced scrutiny due to opaque reserve practices and jurisdictional inconsistencies. Studies by the BIS [7] and the FATF [6] highlight systemic risks arising from undercollateralization, legal ambiguity, and insufficient compliance enforcement. Our protocol introduces a modular reserve transparency framework based on oracle or zk-oracle attestations, enabling verifiable solvency without exposing sensitive balance sheet information.

CBDCs and Centralized Governance. Central bank digital currencies (CBDCs), such as China's e-CNY and the ECB's digital euro, emphasize monetary sovereignty and programmable policy enforcement [**?**]. However, they rely on centralized control structures that may conflict with privacy and innovation goals. In contrast, our protocol is issuer-agnostic, enabling regulated banks and institutions to tokenize deposits on Ethereum under standardized and interoperable rules.

On-Chain Compliance and Privacy. Prior work, including zkKYC systems, Zcash-inspired compliance proofs, and privacy-preserving AML frameworks, explores the intersection of cryptographic privacy and regulatory enforcement. Our approach adopts a narrower and more deployable model based on zero-knowledge membership proofs, enforcing eligibility without on-chain identity disclosure and aligning with institutional regulatory requirements under the GENIUS Act and MiCA.

Smart Contract Standards and Upgradability. Our implementation builds on best practices from OpenZeppelin's contract library, including `Ownable`, `Pausable`, and `AccessControl` modules, to enforce access boundaries and emergency controls. Unlike prior upgradeability frameworks, governance actions in our protocol are constrained by multisignature authorization and timelocks to mitigate administrative capture risks.

Summary. Our work bridges regulatory compliance, reserve-backed financial engineering, and Ethereum-native composability. To our knowledge, this is the first open-source deposit token protocol designed to meet both GENIUS Act and MiCA regulatory requirements while remaining fully deployable on a public blockchain.

13 Future Work

13.1 zkOracles for Privacy-Preserving Reserve Attestations

Zero-knowledge oracles (zkOracles) enable privacy-preserving attestations of off-chain data, such as reserve balances or audit results, without revealing sensitive institutional information on-chain. Instead of publishing raw reserve values, an oracle-backed prover generates a succinct zero-knowledge proof asserting that regulatory constraints (e.g., minimum reserve ratios) are satisfied.

In our protocol, zkOracles are considered as an optional enhancement to the ReserveRegistry, complementing standard oracle-based reserve reporting. This design allows issuers to demonstrate solvency and regulatory compliance while minimizing disclosure of proprietary balance sheet data. Smart contracts verify the submitted proof and enforce protocol invariants without learning the underlying reserve amounts.

Importantly, zkOracles are not used for identity verification or AML/KYC enforcement. Participant eligibility is handled separately via zero-knowledge membership proofs, while reserve sufficiency is enforced through oracle or zk-oracle attestations. This separation of concerns preserves modularity, auditability, and privacy.

Figure 3 illustrates the zkOracle attestation flow, in which off-chain data providers and auditors supply inputs to a prover that generates a zero-knowledge proof. The proof is submitted on-chain and verified by the protocol contracts, enabling compliance enforcement without direct data disclosure.

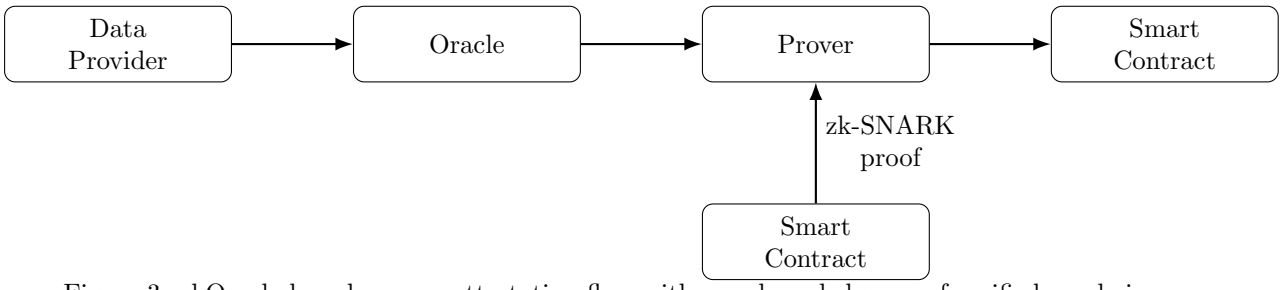

Figure 3: zkOracle-based reserve attestation flow with zero-knowledge proof verified on-chain

The flow illustrated in Figure 3 demonstrates how reserve data or user credentials are verified by off-chain providers, who then generate a zero-knowledge proof. This proof is submitted to a smart contract verifier, which checks its validity without learning the underlying data. This mechanism allows enforcement of on-chain reserve constraints, while participant eligibility is enforced separately via zero-knowledge membership proofs.

13.2 Cross-Chain Interoperability

Institutional deposit tokens must be composable beyond a single blockchain ecosystem to support global settlement infrastructure. Our protocol can be extended using cross-chain bridges to enable transferability across chains like Ethereum, Base, Arbitrum, and international Layer-1 networks.

To achieve secure interoperability, we adopt a modular bridging strategy:

- **Canonical Bridge:** A bank-approved bridge contract deployed on each chain, governed by a multi-signature committee.
- **Message Verification:** Use of LayerZero or Hyperlane for verifiable message passing with replay protection.
- **Reserve Synchronization:** Oracles post reserve updates and token supply snapshots across chains, with zk-proofs for trust minimization.

Let DTK_i represent the deposit token on chain i, and $\mathrm{Bridge}_{i \to j}$ denote the bridge for transferring tokens from chain i to j. The invariant:

$$\sum_i \mathrm{Supply}(\mathrm{DTK}_i) \leq R_t$$

must be enforced globally to prevent over-minting. Cross-chain mint and burn events are subject to timelocks and eligibility verification via zero-knowledge membership proofs, with reserve consistency enforced by oracles.

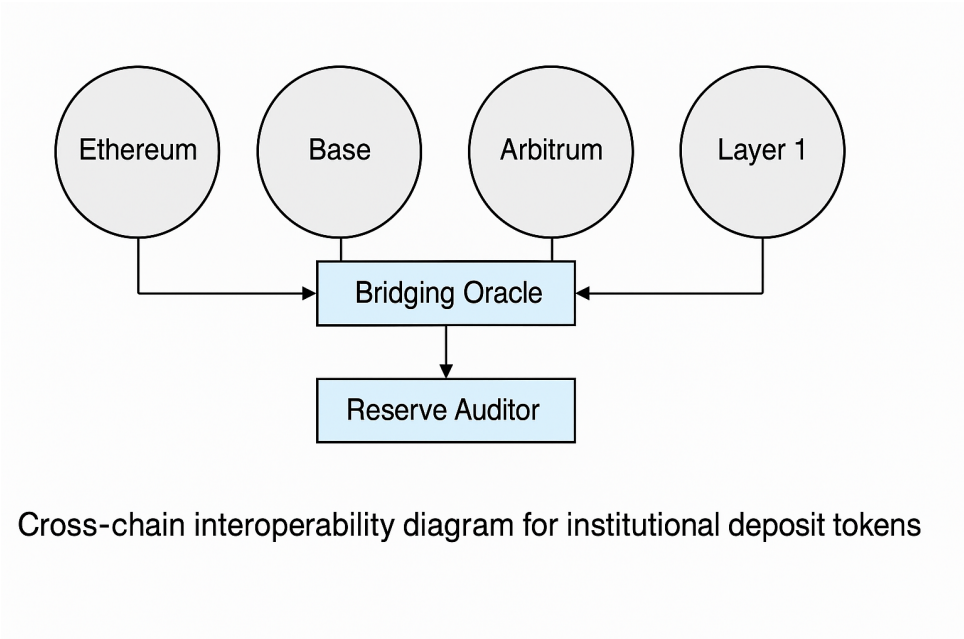

Figure 4: Cross-chain interoperability: Message passing and token bridging via LayerOne endpoints and relayers

This model supports emerging cross-border use cases in regions like Southeast Asia, Latin America, and Sub-Saharan Africa, where mobile-first stablecoin adoption is accelerating.

14 Conclusion

We have presented an Ethereum-based protocol for institutional deposit tokens that combines regulatory compliance, strong cryptographic guarantees, and modular smart contract design. Our architecture—comprising the DepositToken, ReserveRegistry, RedemptionModule, and GovernanceController—enables banks and regulated institutions to issue fully backed digital deposits on public blockchain infrastructure with real-time reserve validation and role-based governance.

The protocol enforces AML/KYC compliance through privacy-preserving zero-knowledge membership proofs, while reserve sufficiency is ensured via oracle or zk-oracle attestations. Emergency circuit breakers and transparent governance mechanisms provide safeguards against operational and regulatory risks. Through formal modeling and performance benchmarks on Ethereum Sepolia, we demonstrate the protocol's security, composability, and readiness for institutional deployment.

This work bridges the gap between private, institution-issued digital settlement systems and open, interoperable blockchain finance, enabling programmable, regulation-aligned deposit tokens without sacrificing user privacy or Ethereum-native composability. Future work includes deeper zkOracle integration, cross-chain liquidity management, and collaboration with regulators and financial institutions to pilot real-world deployments.

References

[1] Ethereum Foundation, *ERC-20: Token Standard*, 2015.
https://eips.ethereum.org/EIPS/eip-20

[2] Ethereum Foundation, *ERC-4626: Tokenized Vault Standard*, 2022.
https://eips.ethereum.org/EIPS/eip-4626

[3] Ethereum Foundation, *ERC-3643: Permissioned Token Standard (formerly T-REX)*, 2021.
https://eips.ethereum.org/EIPS/eip-3643

[4] U.S. Senate, *GENIUS Act: Stablecoin Regulatory Framework*, 2025.
https://www.senate.gov

[5] European Union, *Markets in Crypto-Assets (MiCA) Regulation*, 2024.
https://eur-lex.europa.eu

[6] Financial Action Task Force (FATF), *Guidance for a Risk-Based Approach to Virtual Assets and VASPs*, 2021.
https://www.fatf-gafi.org

[7] Bank for International Settlements, *Stablecoins and CBDCs: Implications for Financial Stability*, 2022.
https://www.bis.org/publ/bppdf/bispap121.htm

[8] Centre Consortium, *USDC Attestation Reports*, Monthly Reports by Grant Thornton LLP.
https://www.centre.io/usdc-transparency

[9] CoinGecko, *Top Stablecoins by Market Capitalization*, June 2025.
https://www.coingecko.com/en/categories/stablecoins

[10] JPMorgan, *JPM Coin: Digitizing Payments*, 2020.
https://www.jpmorgan.com/solutions/jpm-coin

[11] Chainlink Labs, *Decentralized Oracle Networks*, 2023.
https://chainlink.network

[12] Jens Groth, *On the Size of Pairing-Based Non-Interactive Arguments*, Eurocrypt 2016.
https://eprint.iacr.org/2016/260

[13] Ariel Gabizon, Zachary J. Williamson, Oana Ciobotaru, *PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*, 2019.
https://eprint.iacr.org/2019/953